Project Title

Community Detection Platform

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
G .SATWIKA (2520090111)
B .SNIGDHA (2520090160)

Team Number: 30

Section: 10

Under the guidance of

MS.CHANDUSHA

Ass.prof, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING /

COMPUTER SCIENCE AND INFORMATION

TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE

DECLARATION

The Mini Project Report entitled "**Project Title"** is a record of Bonafide work of G .SATWIKA (2520090111), B .SNIGDHA (2520090160)**.**submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

G .SATWIKA (2520090111)
B .SNIGDHA (2520090160)

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Project Title"** is a bonafide work done and submitted by G .SATWIKA (2520090111), B .SNIGDHA (2520090160)

. in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, MS.CHANDUSHA, Ass.prof, CS&IT

which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

G .SATWIKA (2520090111),
B .SNIGDHA (2520090160)

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

The **Community Insights – Community Detection Platform** is a full-stack web application developed to analyze social and communication networks and identify groups of users with strong interaction patterns. The primary objective of this project is to apply Data Structures and Algorithms (DSA) to real-world network analysis and provide meaningful insights through an interactive dashboard. The system represents users as nodes and their interactions as weighted edges in an undirected graph, where edge weights indicate interaction strength. Users can upload CSV files containing interaction data, which are validated and processed by the backend.

The application is developed using Next.js, TypeScript, React, and Tailwind CSS, with Next.js API routes handling backend processing. A weighted adjacency-list data structure is used to represent the graph efficiently. Graph traversal algorithms, including Breadth-First Search (BFS) and Depth-First Search (DFS), are implemented to explore network connections. The Louvain community detection algorithm uses modularity optimization to identify groups with stronger internal connections. The system also calculates important network metrics, including the number of users, interactions, average degree, network density, community sizes, total interaction strength, and modularity score.

The results are presented through an interactive network visualization, analytical charts, community summaries, and searchable user tables. The platform also supports exporting analysis results in CSV and JSON formats. By combining graph algorithms with a user-friendly web interface, Community Insights demonstrates the practical application of DSA in social network analysis, audience segmentation, marketing strategy, and community-based recommendations. Its serverless-compatible architecture enables deployment on Vercel without requiring a separate Java backend.

Team Name:
30

Team Logo (if any):

Team Members:

G .SATWIKA (2520090111),
B .SNIGDHA (2520090160)

2. INTRODUCTION

In today's digital world, social media platforms, online communities, and communication networks generate large amounts of interaction data every day. Understanding how users interact with one another is important for identifying common interests, analyzing relationships, improving recommendations, and developing effective marketing strategies. However, manually analyzing large networks is difficult because users may have numerous connections and complex interaction patterns. Community detection provides a solution by identifying groups of users who interact more frequently or strongly with one another than with the rest of the network.

The **Community Insights – Community Detection Platform** is a full-stack web application developed to analyze weighted social networks using Data Structures and Algorithms (DSA). The system represents users as nodes and their interactions as weighted edges in an undirected graph. It uses an adjacency-list representation for efficient graph storage and implements Breadth-First Search (BFS) and Depth-First Search (DFS) for graph traversal. The Louvain community detection algorithm is used to identify groups by optimizing modularity, while network metrics provide insights into connectivity and community structure. Users can upload interaction data through CSV files and view the analysis results using an interactive dashboard containing network visualizations, charts, community summaries, and user statistics.

The application is developed using Next.js, TypeScript, React, and Tailwind CSS, with Next.js API routes processing uploaded data and performing graph analysis. Its architecture supports deployment on Vercel without requiring a separate Java backend. The main objective is to demonstrate how fundamental DSA concepts can solve practical network analysis problems through an accessible web interface. This project report presents the system's objectives, requirements, design, algorithms, implementation, results, and potential applications in social network analysis, audience segmentation, and targeted marketing.

3. LITERATURE SURVEY

4. 1. Gephi: Gephi is a network visualization tool used to analyze relationships between users. It helps identify communities and display connections through graphs. Our project uses a similar approach with a web-based dashboard.
5. Website: https://gephi.org/
6. 2. Neo4j: Neo4j is a graph database platform that supports community detection and network analysis. It provides algorithms such as Louvain to identify closely connected groups. Our project uses Louvain for community detection.
7. Website: https://neo4j.com/
8. 3. NetworkX: NetworkX is a Python library used to create and analyze graphs. It supports BFS, DFS, and community detection algorithms. Our project implements similar graph concepts using TypeScript.
9. Website: https://networkx.org/
10. 4. Louvain Algorithm: The Louvain algorithm identifies communities by optimizing modularity. It groups users based on the strength of their connections. Our project uses this algorithm to detect communities in weighted networks.
11. Research paper: https://doi.org/10.1088/1742-5468/2008/10/P10008
12. 5. Cytoscape: Cytoscape is a software platform used to visualize and analyze networks. It displays nodes and edges to help users understand relationships. Our project also provides interactive network visualization.
13. Website: https://cytoscape.org/
14. 6. igraph: igraph is a graph analysis library used to study network structures. It supports graph algorithms, community detection, and network metrics. Our project uses similar techniques to analyze user interactions.
15. Website: https://igraph.org/

16. Problem Statement and Objectives

Problem Statement: To develop a platform that identifies communities in complex networks using graph algorithms and visualizes relationships between users.

Problem Description

The project analyzes user interactions represented as a graph to detect closely connected communities. It uses graph algorithms and community detection techniques to display network structure and key metrics.

Objectives – Identify the Project's Purpose

- To represent user interactions using graphs with weighted edges.
- To implement BFS and DFS for graph traversal.
- To detect communities using the Louvain algorithm.
- To calculate network metrics and community statistics.
- To visualize the network and detected communities through an interactive dashboard.
- To help users understand relationships and patterns in complex networks.

17. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i5 or equivalent
- RAM: Minimum 8 GB
- Storage: Minimum 256 GB HDD/SSD

Software Requirements

- Operating System: Windows 10/11 or Linux
- Frontend: Next.js, React, TypeScript
- Backend: Next.js API Routes (Node.js)
- Tools: Visual Studio Code and Node.js/npm

18. IMPLEMENTATION

Module 1: Graph Traversal Using BFS and DFS — CO1

Course Outcome Mapping: CO1 – Advanced algorithm strategies.

```
export function bfs(
  graph: Map<string, string[]>,
  start: string
): string[] {
  const visited = new Set<string>();
  const queue: string[] = [start];
  const result: string[] = [];

  visited.add(start);

  while (queue.length > 0) {
    const node = queue.shift()!;
    result.push(node);

    for (const neighbor of graph.get(node) ?? []) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
      }
    }
```

Module 2: String Pattern Matching Using Rabin–Karp — CO2

Course Outcome Mapping: CO2 – Apply linear-time string algorithms, including Rabin–Karp, to solve pattern-matching problems.

```
export function rabinKarp(
  text: string,
  pattern: string
): number[] {
  const positions: number[] = [];
  const m = pattern.length;

  if (m === 0 || m > text.length) return positions;

  for (let i = 0; i <= text.length - m; i++) {
    const substring = text.slice(i, i + m);
```

```
    if (substring === pattern) {
      positions.push(i);
    }
  }

  return positions;
}
```

Module 3: Network Flow Using Ford–Fulkerson — CO4

Course Outcome Mapping: CO4 – Apply network-flow algorithms to model and solve capacity-constrained problems.

```
export function fordFulkerson(
  capacity: number[][],
  source: number,
  sink: number
): number {
  const residual = capacity.map(row => [...row]);
  const n = residual.length;
  let maxFlow = 0;

  function dfs(
    u: number,
    visited: boolean[],
    flow: number
  ): number {
    if (u === sink) return flow;
    visited[u] = true;

    for (let v = 0; v < n; v++) {
      if (!visited[v] && residual[u][v] > 0) {
        const pushed = dfs(
          v,
          visited, Math.min(flow, residual[u][v])
        );

        if (pushed > 0) {
          residual[u][v] -= pushed;
          residual[v][u] += pushed;
          return pushed;
        }
      }
    }

    return 0;
  }

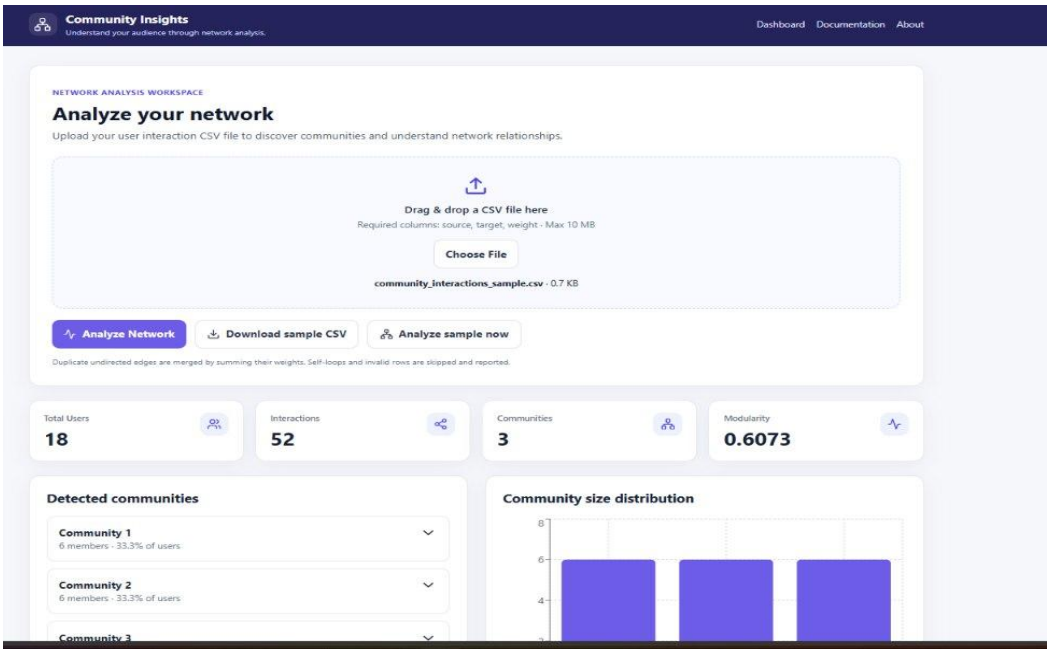
  while (true) {
    const visited = Array(n).fill(false);
    const flow = dfs(source, visited, Infinity);

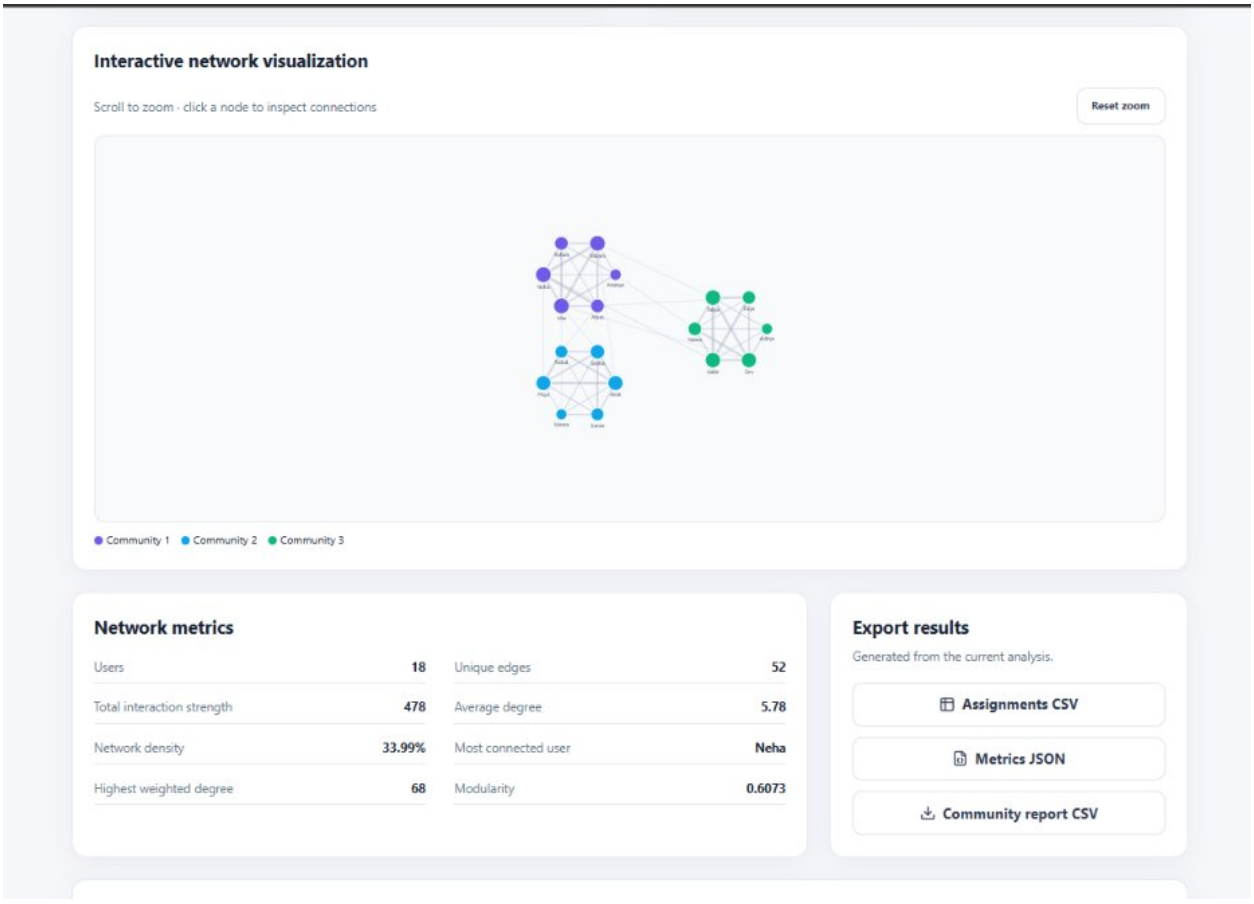
    if (flow === 0) break;
    maxFlow += flow;
```

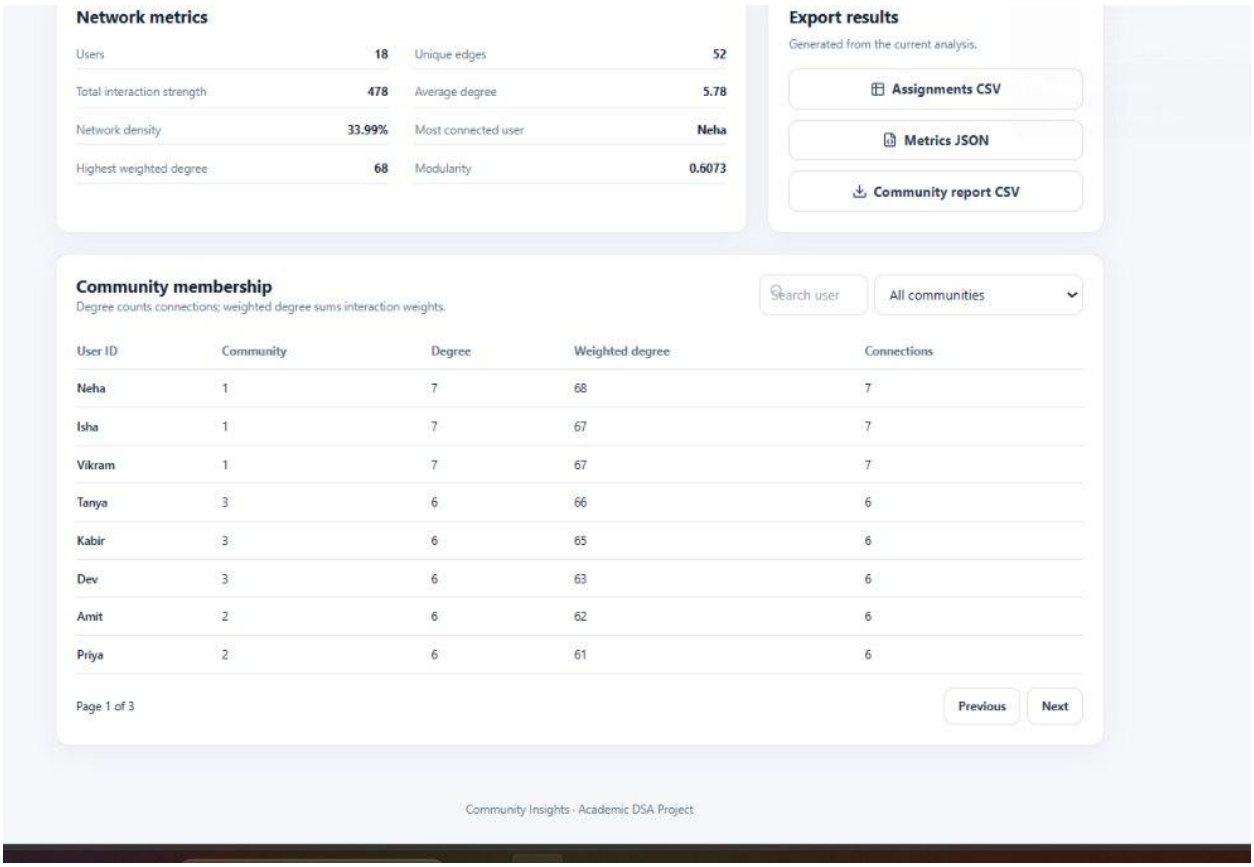
```
  }

  return maxFlow;
}
```

19. RESULTS & DISCUSSION






Network metrics
Users 18
Unique edges 52
Total interaction strength 478
Average degree 5.78
Network density 33.99%
Most connected user Neha
Highest weighted degree 68
Modularity 0.6073
Export results
Generated from the current analysis.
Assignments CSV
Metrics JSON
Community report CSV
Community membership
Degree counts connections; weighted degree sums interaction weights.
Search user
All communities
User ID Community Degree Weighted degree Connections
Neha 1 7 68 7
Isha 1 7 67 7
Vikram 1 7 67 7
Tanya 3 6 66 6
Kabir 3 6 65 6
Dev 3 6 63 6
Amit 2 6 62 6
Priya 2 6 61 6
Page 1 of 3
Previous
Next
Community Insights · Academic DSA Project

20. CONCLUSION & FUTURE SCOPE

Conclusion

The Community Detection Platform helps analyze user relationships using graph data structures and algorithms. It aims to identify communities, explore connections, calculate network metrics, and visualize network patterns through an interactive dashboard. The project provides a useful way to understand complex networks and their relationships.

Future Scope

1. Short-Term Enhancements

- Improve the user interface and network visualization.
- Add more sample datasets and user interactions.
- Improve graph traversal and network analysis features.

2. Medium-Term Enhancements

- Implement a complete Louvain community detection algorithm.
- Support uploading and analyzing real-world datasets.
- Add advanced network metrics and community comparisons.

3. Long-Term Enhancements

- Apply machine learning to predict potential communities.
- Analyze large-scale social and communication networks.
- Develop real-time community detection and monitoring features.

.

21. REFERENCES

1 • Gephi – Network Visualization and Analysis. https://gephi.org/

- Neo4j – Graph Database and Network Analysis. https://neo4j.com/
- NetworkX – Python Library for Graph Analysis. https://networkx.org/
- Blondel, V. D., et al. (2008). Fast unfolding of communities in large networks. https://doi.org/10.1088/1742-5468/2008/10/P10008
- Cytoscape – Network Visualization Platform. https://cytoscape.org/
- igraph – Network Analysis Software. https://igraph.org/
- Traag, V. A., et al. (2019). From Louvain to Leiden: guaranteeing well-connected communities. https://doi.org/10.1038/s41598-019-41695-z
- NetworkX Documentation – Community Detection Algorithms. https://networkx.org/documentation/stable/reference/algorithms/community.html
- Next.js Documentation. https://nextjs.org/docs